In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix,accuracy_score,precision_score,recall_
from sklearn.preprocessing import StandardScaler
```

In [3]:
```
df = pd.read_csv('/home/backup/ml_datasets/pima_indians_diabetes.csv');
print(df.head(20))
```

```
    preg   plas  pres  skin   insu  mass   pedi   age            class
0    6.0  148.0  72.0  35.0    0.0  33.6  0.627  50.0  tested_positive
1    1.0   85.0  66.0  29.0    0.0  26.6  0.351  31.0  tested_negative
2    8.0  183.0  64.0   0.0    0.0  23.3  0.672  32.0  tested_positive
3    1.0   89.0  66.0  23.0   94.0  28.1  0.167  21.0  tested_negative
4    0.0  137.0  40.0  35.0  168.0  43.1  2.288  33.0  tested_positive
5    5.0  116.0  74.0   0.0    0.0  25.6  0.201  30.0  tested_negative
6    3.0   78.0  50.0  32.0   88.0  31.0  0.248  26.0  tested_positive
7   10.0  115.0   0.0   0.0    0.0  35.3  0.134  29.0  tested_negative
8    2.0  197.0  70.0  45.0  543.0  30.5  0.158  53.0  tested_positive
9    8.0  125.0  96.0   0.0    0.0   0.0  0.232  54.0  tested_positive
10   4.0  110.0  92.0   0.0    0.0  37.6  0.191  30.0  tested_negative
11  10.0  168.0  74.0   0.0    0.0  38.0  0.537  34.0  tested_positive
12  10.0  139.0  80.0   0.0    0.0  27.1  1.441  57.0  tested_negative
13   1.0  189.0  60.0  23.0  846.0  30.1  0.398  59.0  tested_positive
14   5.0  166.0  72.0  19.0  175.0  25.8  0.587  51.0  tested_positive
15   7.0  100.0   0.0   0.0    0.0  30.0  0.484  32.0  tested_positive
16   0.0  118.0  84.0  47.0  230.0  45.8  0.551  31.0  tested_positive
17   7.0  107.0  74.0   0.0    0.0  29.6  0.254  31.0  tested_positive
18   1.0  103.0  30.0  38.0   83.0  43.3  0.183  33.0  tested_negative
19   1.0  115.0  70.0  30.0   96.0  34.6  0.529  32.0  tested_positive
```

In [5]:
```
df['class'] = df['class'].map({'tested_positive':1, 'tested_negative' :0})
X = df.drop('class', axis = 1)
y = df['class']
```

In [6]:
```
print(df.head(20))
```

```
    preg   plas  pres  skin   insu  mass   pedi   age  class
0    6.0  148.0  72.0  35.0    0.0  33.6  0.627  50.0      1
1    1.0   85.0  66.0  29.0    0.0  26.6  0.351  31.0      0
2    8.0  183.0  64.0   0.0    0.0  23.3  0.672  32.0      1
3    1.0   89.0  66.0  23.0   94.0  28.1  0.167  21.0      0
4    0.0  137.0  40.0  35.0  168.0  43.1  2.288  33.0      1
5    5.0  116.0  74.0   0.0    0.0  25.6  0.201  30.0      0
6    3.0   78.0  50.0  32.0   88.0  31.0  0.248  26.0      1
7   10.0  115.0   0.0   0.0    0.0  35.3  0.134  29.0      0
8    2.0  197.0  70.0  45.0  543.0  30.5  0.158  53.0      1
9    8.0  125.0  96.0   0.0    0.0   0.0  0.232  54.0      1
10   4.0  110.0  92.0   0.0    0.0  37.6  0.191  30.0      0
11  10.0  168.0  74.0   0.0    0.0  38.0  0.537  34.0      1
12  10.0  139.0  80.0   0.0    0.0  27.1  1.441  57.0      0
13   1.0  189.0  60.0  23.0  846.0  30.1  0.398  59.0      1
14   5.0  166.0  72.0  19.0  175.0  25.8  0.587  51.0      1
15   7.0  100.0   0.0   0.0    0.0  30.0  0.484  32.0      1
16   0.0  118.0  84.0  47.0  230.0  45.8  0.551  31.0      1
17   7.0  107.0  74.0   0.0    0.0  29.6  0.254  31.0      1
18   1.0  103.0  30.0  38.0   83.0  43.3  0.183  33.0      0
19   1.0  115.0  70.0  30.0   96.0  34.6  0.529  32.0      1
```

In [7]:
```
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)

print(f"X_train shape: {X_train.shape[0]}")
print(f"X_test shape: {X_test.shape[0]}")
print(f"y_train shape: {y_train.shape[0]}")
print(f"y_test shape: {y_test.shape[0]}")
```

```
X_train shape: 614
X_test shape: 154
y_train shape: 614
y_test shape: 154
```

In [8]:
```
model = LogisticRegression(max_iter=1000,random_state=42)
model.fit(X_train,y_train)
print(f"Intercept: {model.intercept_}")
print(f"Coefficient: {model.coef_}")
y_pred = model.predict(X_test)
```

```
Intercept: [-9.00638276]
Coefficient: [[ 0.06439821  0.03409515 -0.0138825   0.00329109 -0.00180359  0.1026105
5
   0.62663309  0.03708617]]
```

In [9]:
```
print("\n--DETAILS--")
print(f"Accuracy:{accuracy_score(y_test, y_pred ):.4f}")
print(f"Precision:{precision_score(y_test, y_pred ):.4f}")
print(f"Recall:{recall_score(y_test, y_pred ):.4f}")
print(f"F1_score:{f1_score(y_test, y_pred ):.4f}")
```

```
--DETAILS--
Accuracy:0.7468
Precision:0.6379
Recall:0.6727
F1_score:0.6549
```

In [10]:
```
#preprocessing
scalar= StandardScaler()
scalar.fit(X_train)
X_train_scaled=scalar.transform(X_train)
X_test_scaled=scalar.transform(X_test)
print("Mean/std used for scaling:",scalar.mean_,scalar.scale_)
```

```
Mean/std used for scaling: [  3.74267101 120.85504886  69.41530945  20.3990228   81.4
3811075
  31.98338762   0.46916775  32.90716612] [  3.31056497  32.00895893  18.49751705  15.
42140098 116.14014299
   7.73431907   0.33657233  11.49406506]
```

In [11]:
```
model_scaled = LogisticRegression(max_iter=1000,random_state=42)
model_scaled.fit(X_train,y_train)
print(f"Intercept: {model_scaled.intercept_}")
print(f"Coefficient: {model_scaled.coef_}")
y_pred_scaled = model_scaled.predict(X_test)
```

```
Intercept: [-0.88612313]
Coefficient: [[ 0.21255833  1.07113231 -0.24784154  0.04569683 -0.2008266   0.7781515
1
   0.23060673  0.42100175]]
```

In [12]:
```
print("\n--DETAILS--")
print(f"Accuracy:{accuracy_score(y_test, y_pred_scaled ):.4f}")
```

```
print(f"Precision:{precision_score(y_test, y_pred_scaled ):.4f}")
print(f"Recall:{recall_score(y_test, y_pred_scaled ):.4f}")
print(f"F1_score:{f1_score(y_test, y_pred_scaled ):.4f}")
```

```
--DETAILS--
Accuracy:0.7532
Precision:0.6491
Recall:0.6727
F1_score:0.6607
```

In [13]:

```
data = {'Model' :['UNSCALED','SCALED'],
        'Accuracy' : [accuracy_score(y_test, y_pred ), accuracy_score(y_test, y_pred
df = pd.DataFrame(data)
print(df)
```

```
      Model  Accuracy
0  UNSCALED  0.746753
1    SCALED  0.753247
```

In [ ]: